

RANsliceOpt AI: SLA-Constrained Soft RAN Slicing via PPO over a Multi-Step MDP

A Browser-Prototyped Architecture with a Deployment Path to Latency-Aware gNB Edge Inference

RANsliceOpt AI Working Draft

2026-08-18

Abstract

RANsliceOpt AI is a browser-based development and simulation platform for AI-assisted soft RAN slicing across enhanced Mobile Broadband (eMBB), Ultra-Reliable Low-Latency Communication (URLLC), and massive IoT/eMTC traffic. Rather than a single-step contextual-bandit formulation, the system models slice resource-envelope adaptation as a multi-step Markov Decision Process (MDP) solved with a compact Proximal Policy Optimization (PPO) actor-critic. The specification separates three control time scales – a fast deterministic MAC scheduler, a locally executed PPO slice-control loop, and a slower supervisory layer that may include a small language model (SLM) – so that latency-critical decisions never depend on a network round trip or on SLM inference. Hard service-level agreement (SLA) protection is enforced by a deterministic safety guard rather than by reward shaping alone, and a net-neutrality constraint excludes application or content identity from state, reward, and priority logic. This paper summarizes the system’s purpose, MDP and PPO design, reward and constraint architecture, deployment workflow, and acceptance-test criteria, and situates the design relative to 3GPP and O-RAN slicing concepts.

1. Introduction

Network slicing partitions a shared radio access network (RAN) into logical services with distinct performance guarantees. 3GPP and O-RAN define the service classes and the management/execution timescale separation that make such slicing standards-compliant, but they do not mandate a specific optimization algorithm for adapting slice resource shares under changing traffic. RANsliceOpt AI addresses this gap for **soft** slicing – dynamic resource sharing with enforceable minimum guarantees and protected reserves, as opposed to static partitioning.

The design targets three simultaneous traffic classes: mainstream eMBB, latency- and reliability-critical URLLC, and high-density, low-rate mMTC. Its primary objective is to improve spectrum and resource efficiency under changing traffic conditions while continuously preserving declared per-slice SLAs. A net-neutrality principle is treated as a first-class design constraint: optimization decisions must derive from service-class technical requirements and declared SLA/QoS parameters, never from application-provider identity, content, or commercial preference.

The reference implementation is a browser prototype in which all state, action, and reward logic is inspectable, reproducible, exportable, and executable offline after initial load. This constraint keeps

the system auditable during development and gives the eventual gNB/O-DU deployment a browser-verifiable reference behavior to match.

2. Three Control Time Scales

A central design decision is the separation of RAN optimization into three timescales, summarized in Table 1. The commonly cited 1 ms URLLC user-plane latency target is **not** interpreted as a requirement that an SLM, a PPO training cycle, or a full management loop complete within 1 ms. Instead, only the compact PPO forward pass and the deterministic MAC scheduler sit in the latency-critical path; the scheduler retains final authority over instantaneous physical resource block (PRB) assignment, while PPO controls resource *envelopes and priorities* rather than PHY/MAC scheduling detail.

Layer	Role	Placement	Latency Principle
Fast MAC loop	Exact UE/PRB assignment, MCS/HARQ, deadline handling	gNB/O-DU	Sub-ms to few-ms, implementation-dependent
PPO slice-control loop	Adjusts slice resource envelopes from current MDP state	gNB/O-DU local inference	Fast enough to influence near-term scheduling; no external round trip
Supervisory AI/training	SLM intent interpretation, policy generation, PPO training/validation	Edge accelerator or management domain	Outside the URLLC critical path

The end-to-end functional flow follows operator intent and SLA/neutrality policy down through an optional SLM supervisory layer, into policy constraints and objective weights, an MDP state builder, the PPO actor/critic, a constrained action, a safety/SLA guard, and finally the real-time MAC scheduler. KPI observations close the loop into the next MDP state, a reward, and a trajectory buffer, repeating over the episode.

3. Traffic Classes and Soft-Slice Intent

Each slice carries a distinct protected metric and optimization behavior (Table 2). Unused protected capacity is borrowable by other slices when the owning slice has no immediate demand, subject to a reclaim rule that lets the protected slice recover capacity rapidly – this is the mechanism by which the system remains “soft” rather than statically partitioned.

Slice	Primary Need	Protected Metric	Optimization Behavior
URLLC	Very low latency, high reliability	Deadline/latency, reliability, queue urgency	Protected reserve; rapid expansion; may preempt elastic eMBB within limits
eMBB	High throughput, user experience	Throughput, cell-edge rate, fairness	Uses mainstream capacity; opportunistic

Slice	Primary Need	Protected Metric	Optimization Behavior
			cally borrows unused reserve
mIoT/eMTC	High device density, efficient data service	Access success, density, resource efficiency	Small guaranteed pool; aggregate scheduling; avoids starving other slices

4. MDP Formulation

State. At decision step t , the state vector represents current radio and resource conditions rather than raw history: total and committed PRBs; per-slice PRB use and requested load; URLLC queue depth, head-of-line delay, deadline margin, and recent reliability outcome; eMBB offered load, mean and 5th-percentile (cell-edge) throughput, and Jain fairness; mIoT/eMTC active-device density and aggregate offered traffic; per-slice spectral-efficiency and CQI summaries; current shares, minimum guarantees, maximum caps, and borrowable capacity; SLA violation flags and distance-to-threshold values; and optional power/energy and morphology/MIMO/duplex context. Derived short-window trends may stand in for raw history when required.

Action. The initial action space is bounded discrete or multi-discrete: each slice independently increases, holds, or decreases its resource envelope, with optional power-envelope and spatial-layer dimensions, and joint actions permitted across slices. Every proposed action is projected onto the feasible set so that total allocation never exceeds physical capacity and all configured minimum guarantees remain satisfied – this projection is the safety guard described in Section 6.

Transition and episode. After an action is applied, the simulator advances one scheduling/control interval; traffic arrivals, channel variation, queue service, borrowing, and SLA outcomes determine the next state. Unlike an earlier single-step contextual-bandit formulation, the episode does not terminate after one decision – the agent learns multi-step consequences across a trajectory. Episodes are configurable by step count or simulated time (100–500 steps in development mode by default), with optional warm-up exclusion from training metrics and terminal conditions on horizon end, unrecoverable SLA violation under stress testing, or explicit reset. Trained parameters and training metadata persist across browser closure when persistence is enabled.

5. PPO Actor-Critic Network

The reference architecture is a compact actor-critic sized for browser execution and eventual local gNB inference: a normalized MDP state vector feeds two hidden layers of 64 nodes each with nonlinear activation, shared or separated between actor and critic pathways subject to benchmarking. The actor head outputs action logits/probabilities over the multi-discrete action space; the critic head outputs a scalar estimate of expected future return from the current state.

Training uses the standard PPO clipped-surrogate objective combined with a value loss and an entropy bonus. The critic’s value estimate anchors an advantage calculation that compares observed outcomes against expectation; PPO then updates the actor while clipping excessively large policy changes, and updates the critic via the value loss. Inference in the latency-critical path is forward-pass only – no backpropagation occurs outside the slower training cadence described in Section 7.

6. Reward and Safety-Constraint Architecture

The reward is a weighted composite,

$$R = w_T T + w_F F + w_{\text{edge}} E_{\text{edge}} + w_{\eta} E_{\eta} + w_U U - P,$$

where T , F , E_{edge} , E_{η} , and U are normalized throughput, fairness, cell-edge performance, resource/energy efficiency, and utilization terms, and P aggregates constraint penalties.

Critically, hard SLA constraints do **not** rely on reward penalties alone. A deterministic safety layer rejects or projects any PPO action that would knowingly violate protected minimums, the URLLC reserve/deadline rule, total PRB conservation, or operator policy, covering: URLLC hard latency/reliability protection with a configurable emergency reserve; an eMBB minimum service floor plus fairness protection; an mMTC minimum access/resource floor under high density; total resource conservation; an optional per-slice maximum against monopolization; and a neutrality constraint prohibiting any content/application-provider identity feature from entering the reward or priority logic. This split – reward shapes efficiency, a hard guard enforces safety – is the mechanism by which the system avoids the classical RL failure mode of an agent trading away a mandatory guarantee to maximize expected return.

7. URLLC Fast Path and Deployment Workflow

The fast path is architected so the compact PPO policy runs local to the gNB/O-DU (or an equivalent edge execution environment), explicitly excluding the SLM: the gNB collects compact state (queues, delay margins, load, SLA headroom); the PPO actor performs a local forward pass proposing envelope changes; the safety guard validates the proposal against reserve, minimum-share, and capacity constraints; the MAC scheduler performs exact UE/PRB assignment via normal real-time logic; measured outcomes update state and the trajectory buffer; and PPO training consumes accumulated trajectories at a slower cadence, with updated parameters validated before activation.

Because only a small forward pass sits in the critical path, a GPU is not a functional requirement for every scheduling decision – CPU, NPU, and GPU/accelerator options are all benchmarking targets, with acceleration becoming attractive only once SLM inference or online PPO training is colocated at the edge. The training workflow (scenario generation, fixed/versioned normalization, trajectory collection, GAE-based advantage computation, PPO minibatch/epoch updates with clipping, held-out evaluation including URLLC stress cases, and rejection of any policy showing SLA regression or instability) is kept fully outside the URLLC path, and continuation is only ever resumed from an explicitly accepted checkpoint. The parallel inference/deployment workflow loads a validated policy at the edge, runs local forward-pass inference, applies deterministic safety projection, hands the envelope to the MAC scheduler, and collects outcomes asynchronously – with no mandatory SLM inference or backpropagation permitted in the fast path.

8. SLM Supervisory Role and Net-Neutrality Enforcement

An optional SLM provides a natural-language operator-intent interface but is barred from scheduling URLLC PRBs directly. Its permitted functions are translating operator intent into approved objective weights and constraints, explaining PPO envelope changes, generating policy profiles for operator review, and summarizing SLA risk and borrowing behavior – it may never bypass the safety guard or introduce content-source or provider-based discrimination.

Net-neutrality is enforced structurally rather than by policy statement alone: the optimizer distinguishes technical service-class differentiation (URLLC receiving priority because its declared SLA differs technically from eMBB) from discriminatory application treatment (two flows with equivalent service requirements must receive equivalent treatment regardless of provider). This is implemented as a hard rule set – no content-identity field in the PPO state, minimum floors and a maximum-consecutive-deprivation limit against starvation, and per-action audit logging of the state variables, constraints, and guard result that produced each decision.

9. Evaluation and Acceptance Criteria

Table 3 summarizes the acceptance tests used to validate an implementation prior to release. Passing all tests requires evidence beyond aggregate KPIs: the PPO-authenticity test specifically requires the training log to demonstrate probability-ratio computation, clipping, critic/value updates, and hidden-layer backpropagation, distinguishing a genuine PPO implementation from a heuristic controller relabeled as RL.

Test	Pass Criterion
Capacity conservation	Applied allocations never exceed available resources
URLLC protection	Stress bursts cannot violate the hard reserve/deadline rule when a feasible action exists
Borrow/reclaim	eMBB borrows idle protected resources; URLLC reclaims per configured rule
PPO authenticity	Log shows probability ratio, clipping, value update, backpropagation
MDP behavior	Actions measurably affect subsequent states/rewards over multiple steps
Persistence	Accepted policy survives browser restart when enabled
Neutrality	No provider/content identity in state, reward, or priority logic
Explainability	Each action shows proposal, guard modification, and principal drivers
Timing	Forward-pass latency distribution measured and reported per target hardware
Reproducibility	Same seed/config/model yields statistically consistent results

Every run additionally records configuration hash/version, random seed, simulator version, PPO architecture and hyperparameters, scenario source (synthetic or carrier-provided), checkpoint identifier and training step count, normalization parameters, hardware timing environment, and the full set of active SLA/neutrality constraints – a provenance record intended to make any reported result independently reproducible.

10. Standards Alignment

3GPP defines differentiated logical networks and SLA/KPI assurance across eMBB, URLLC, and massive IoT service classes (3GPP TS 22.261 / ETSI TS 122 261; 3GPP SA5 slice-assurance framework), and O-RAN's slicing architecture (ETSI TS 104 041; O-RAN WG3 Near-RT RIC/E2 scope) separates near-real-time management-plane optimization from faster in-node execution. RANsliceOpt AI follows this separation directly: supervisory intelligence, including any SLM, may run at slower timescales, while the latency-sensitive compact PPO policy and MAC execution remain local to the gNB/O-DU. The specification does not claim PPO itself is mandated by either standards body – PPO is this platform's proposed optimization method, and standards alignment concerns the service classes, slice-assurance concepts, interface/timescale separation, and deployability rather than the learning algorithm. 3GPP TR 37.910 (NR/LTE user-plane latency evaluation for URLLC) grounds the deadline and reliability parameters used in the URLLC protected-reserve logic.

11. Development Phases and Conclusion

Implementation proceeds in seven phases: (1) browser MDP environment with three slices, a deterministic baseline scheduler, and KPI instrumentation; (2) true actor-critic PPO with the 64-node/two-hidden-layer baseline and training diagnostics; (3) the hard SLA safety guard, borrowing/reclaim, and neutrality audit; (4) URLLC fast-path timing instrumentation and an inference-only mode; (5) persistence, CSV carrier input, a stochastic scenario generator, and model provenance; (6) an optional SLM supervisory interface; and (7) an edge deployment benchmark comparing CPU, NPU, and GPU/accelerator execution, with quantization and a gNB/O-DU integration interface as needed.

The governing design decisions – multi-step MDP over single-step contextual bandit, genuine PPO with backpropagation and clipped updates, PPO envelope control paired with deterministic MAC scheduling authority, hard-constraint URLLC protection rather than reward weighting alone, reclaimable borrowing, and empirically benchmarked (not assumed) inference latency – together define a system that pursues spectrum efficiency without trading away declared SLAs or net-neutrality guarantees. The browser prototype exists to make every one of these claims independently inspectable before any component is proposed for a live gNB/O-DU deployment.

References

1. 3GPP TS 22.261 / ETSI TS 122 261, *Service requirements for the 5G system*.
2. 3GPP SA5, *Network slice management overview and SLA/KPI assurance framework*.
3. ETSI TS 104 041, *O-RAN Slicing Architecture*.
4. O-RAN WG3, *Near-Real-time RAN Intelligent Controller and E2 Interface scope*.
5. 3GPP TR 37.910, *Study on evaluation of NR/LTE user-plane latency for URLLC*.